

Thesis

Valentin Falzberger

July 2026

1 Introduction

Anomaly detection aims to find data that is out of place, inconsistent or different than usual or what is expected. This can be done and finds practical application in a lot of different contexts such as medical data [Tschuchnig et al. 2021], fraud detection [Zakaria et al. 2025], cybersecurity [Rajendran et al. 2024] or quality control [Aqeel et al. 2024]. One big issue in automated quality control is the required training of models on how a product is supposed to look like using good images. With recent vision-language segmentation models being able to find objects in images without pre-training just from text prompts, new opportunities arise. The goal of this work is to evaluate the capability of the Segment Anything Model 3 (SAM 3) to detect anomalies in industrial products using a zero-shot approach with simple text prompts. For this purpose, the MVTec AD dataset, containing several different categories, error types and pixel-accurate ground truth masks, is used.



2 Related Work

2.1 SAM 3

Segment Anything Model (SAM) 3 is a model that can segment objects in images or videos. This means the model will highlight those parts of the image or video, that fit the input. As an input, this model takes concept prompts, in the form of short phrases or image exemplars or visual prompts like bounding boxes, masks or points. Additionally, an image or video has to be given as input, on which the segmentation will be done. As a result, SAM 3 will return the found objects in form of masks, together with a probability score for each object, indicating how confident the model is in the object fitting the prompt. Figure 1 illustrates the architecture of the model, where first the image and the prompt are encoded using a Perception Encoder (PE) [Bolya et al. 2025] and objects are detected and segmented by the detector, returning object masks. The tracker and memory bank are used for video segmentation, keeping track of masks across frames. For more details on the implementation see Carion et al. 2025, pages 3–5 and 19–23.

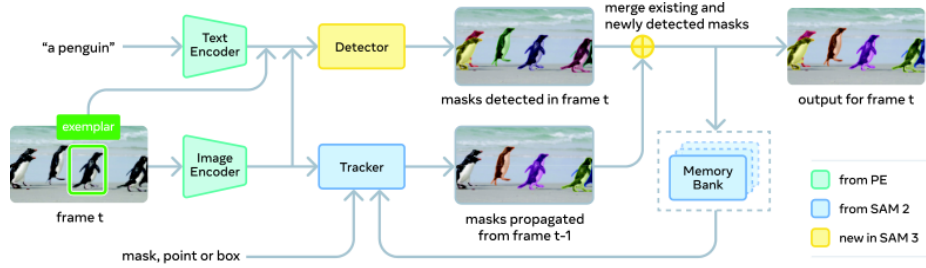


Figure 1: Architecture of SAM 3. From "SAM 3: Segment Anything with Concepts", by N. Carion, 2025, p. 4. <https://arxiv.org/abs/2511.16719> 

To show the capabilities of SAM 3, the authors say:

"To advance PCS [Promptable Concept Segmentation], we build a scalable data engine that produces a high-quality dataset with 4M unique concept labels, including hard negatives, across images and videos. [...] SAM 3 *doubles the accuracy* of existing systems in both image and video PCS, and improves previous SAM capabilities on visual segmentation tasks." [Carion et al. 2025, p. 1]

2.2 SAM in anomaly detection

There have been different approaches to utilize previous iterations of SAM in anomaly detection, most of which are using other models or frameworks in addition. One of those approaches combined SAM together with CLIP (Contrastive Language-Image Pre-Training) [Li et al. 2024], with CLIP providing a rough segmentation and SAM using the resulting point and bounding box prompts to achieve better results. A rough overview can be seen in Figure 2.

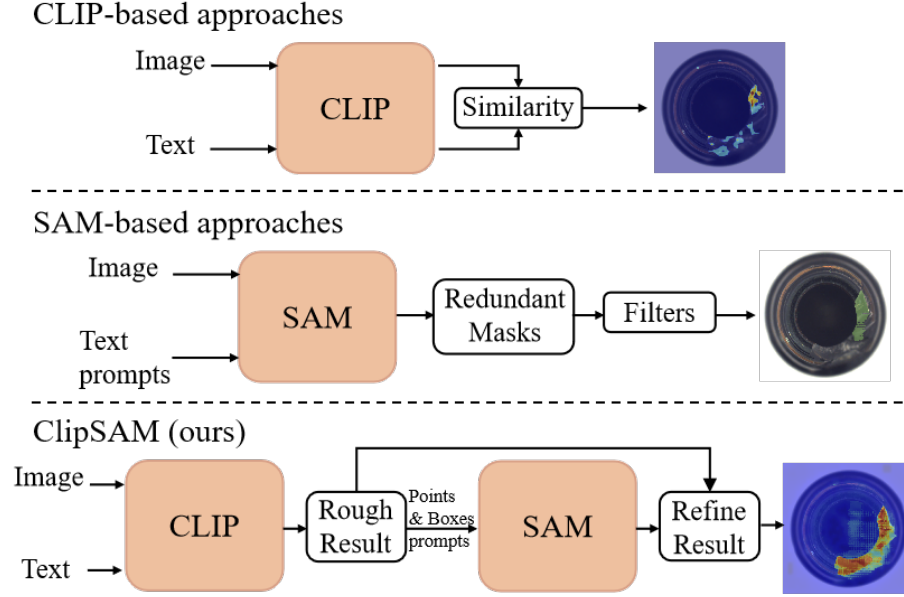


Figure 2: Structure of ClipSAM, compared to approaches for the individual models. From "ClipSAM: CLIP and SAM Collaboration for Zero-Shot Anomaly Segmentation", by S. Li, 2024, Neurocomputing, Vol. 618, p. 2

Another approach was to combine SAM with logic anomaly detection [Peng et al. 2025]. With this, they used feature maps of images together with the object masks from SAM to create object feature maps. The objects are then matched and used to detect logical and structural anomalies. Figure 3 shows an overview of all the steps. They first create feature maps for normal images to build a template bank. For the query image there is also a feature map extracted and reference images are taken from the templates, based on a nearest neighbor search. Both the query image and the reference image are input into SAM, to get object masks. In the last stage, the two object feature maps are compared and matched. Matched and unmatched objects both are relevant for the anomaly scores, making it possible to detect both logical and structural anomalies.

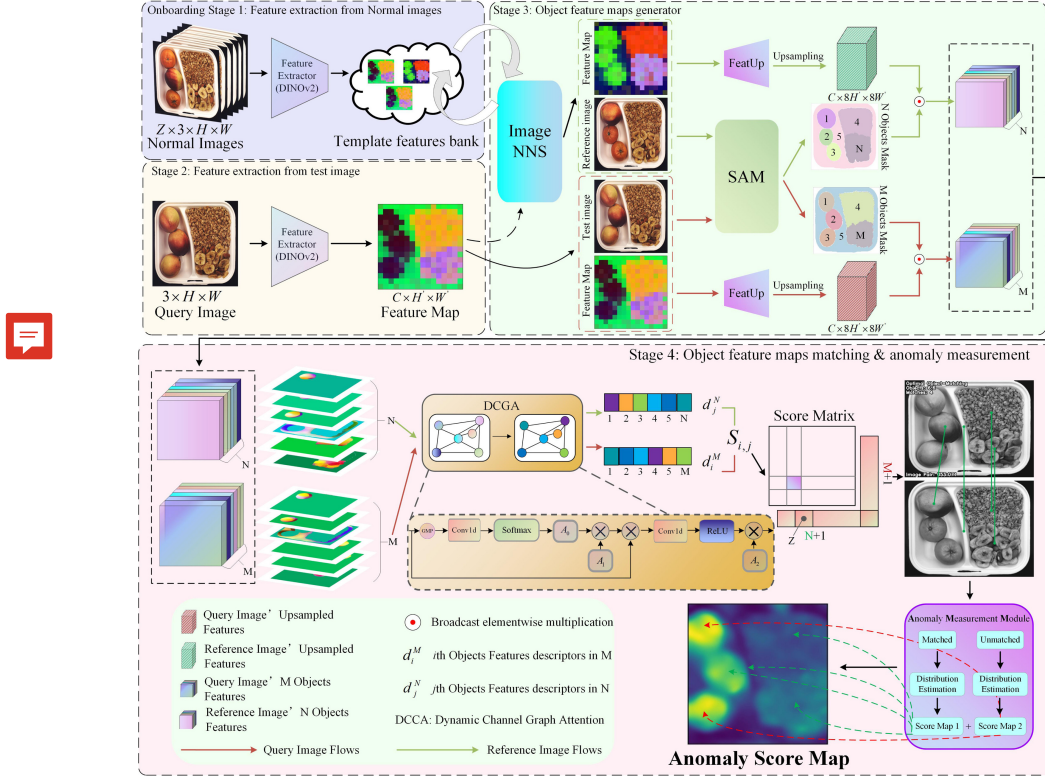


Figure 3: Overview of the SAM-LAD approach. From "SAM-LAD: Segment Anything Model meets zero-shot logic anomaly detection", by Y. Peng, 2025, Knowledge-Based Systems, Vol. 314, p. 4

Yang et al. 2024 used SAM, adding a Self-Perception Tuning (SPT) method, aiming to improve the models performance in the industrial anomaly domain. They first generate a coarse draft of a mask, refining it later. To give the encoder a better understanding of anomalies, they also use a Visual-Relation-Aware Adapter (VRA-Adapter) to get relations of the different image regions. As Figure 4 illustrates, other approaches use the encoded image and prompt directly to create masks, while here the VRA-Adapter is applied to both the refine stage and the draft stage, enhancing the perception ability of SAM.

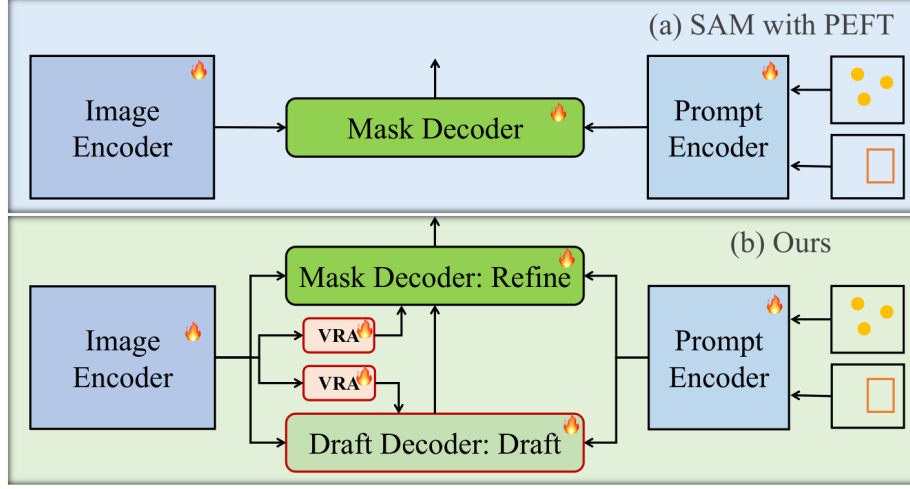


Figure 4: Overview of the SPT approach. From "Promptable Anomaly Segmentation with SAM Through Self-Perception Tuning", by H. Yang, 2024, p. 2, <https://arxiv.org/abs/2411.17217>

While most other research, like the above mentioned, mostly focuses on the point and bounding box prompts SAM accepts, this work takes a look at how effective the new version of SAM 3 can be at zero shot anomaly detection using only text prompts.

3 Methodology

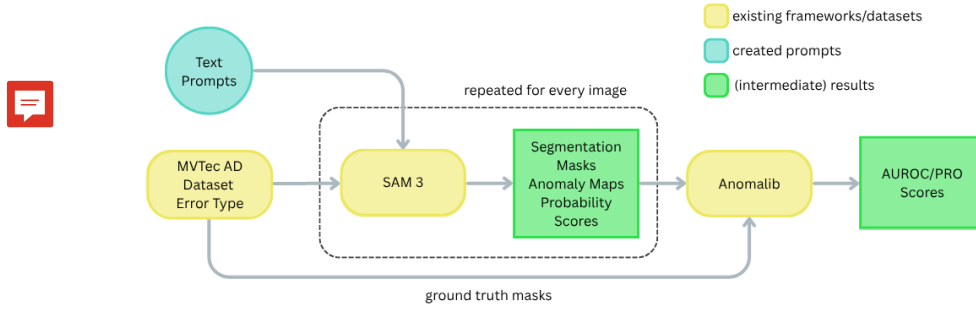


Figure 5: Workflow of the experiment.

For this experiment, the MVTec AD dataset [Bergmann et al. 2021] was used, one of the standard benchmark datasets in regard to industrial anomaly detec-

tion. The dataset contains over 5000 images and almost 70 error types across 15 categories. While there are also some mixed error types included, those left out for this experiment. For every category, there are also good images, one set for training, one for testing, the latter being used in this work.

Table 1: Amount of images in the MVTec dataset, together with the specific prompts per category and error type. For every error type generic prompts "damage", "damaged" and "broken" were also used.



Begin of Table					
category	error type	nr. of prompts	specific prompts	nr. of faulty images	nr. of good images
bottle		15		63	20
	broken_large	5	broken_bottle, broken_glass	20	20
	broken_small	5	broken_bottle, broken_glass	22	20
	contamination	5	contaminated, foreign_objects	21	20
cable		39		92	58
	bent_wire	5	bent_wire, malformed_wire	13	58
	cable_swap	5	cables_with_same_colors, swapped_cables	12	58
	cut_inner_insulation	5	cut_insulation, damaged_insulation	14	58
	cut_outer_insulation	5	cut_insulation, damaged_insulation	10	58
	missing_cable	6	empty_cable, hole, missing_cable	12	58
	missing_wire	7	empty_cable, empty_wire, hole, missing_wire	10	58
	poke_insulation	6	damaged_insulation, hole_in_insulation, poke_insulation	10	58
		27		109	23
capsule	crack	6	crack, cracked, cracked_capsule	23	23
	faulty_imprint	5	faulty_imprint, faulty_text	22	23
	poke	6	damaged_capsule, hole, poke	21	23

Continuation of Table 1					
category	error type	nr. of prompts	specific prompts	nr. of faulty images	nr. of good images
carpet	scratch	5	scratch, scratched_capsule	23	23
	squeeze	5	squeezed, squeezed_capsule	20	23
	color	31		89	28
		7	color, colored_strokes, stain, stains	19	28
		7	cut, cuts, hole, tear	17	28
		6	cut, hole, tear	17	28
	metal_contamination	6	contaminated, foreign_objects, metal_contamination	17	28
grid	thread	5	loose_thread, thread	19	28
	bent	28		57	21
		6	bends, bent, bent_metal	12	21
	broken	5	broken_metal_grid, damaged_metal	12	21
	glue	6	glue, glue_drop, liquid	11	21
	metal_contamination	6	excess_metal, extra_metal, metal_contamination	11	21
hazelnut	thread	5	thread, wool	11	21
	crack	35		70	40
		7	crack, cracked, hole	18	40
	cut	9	cut, hole, notch, scratch, split	17	40
	hole	8	crack, cracked, hole, holes	18	40
	print	11	color, letter, letters, print, text, white_color, white_letters, white_print	17	40
leather	color	30		92	32
		6	color, colored_strokes, stains	19	32
	cut	6	cut, scratch, split	19	32
	fold	6	crease, dents, fold	17	32
	glue	6	glue, glue_drop, liquid	19	32
	poke	6	dents, hole, poke	18	32

Continuation of Table 1					
category	error type	nr. of prompts	specific prompts	nr. of faulty images	nr. of good images
metal_nut		22		93	22
	bent	5	bent, bent_metal	25	22
	color	5	color, colored_strokes	22	22
	flip	5	flip, flipped_nut	23	22
	scratch	7	cut, scratch, scratched, scratches	23	22
pill		35		141	26
	color	6	color, colored_dots, colored_strokes	25	26
	contamination	6	contamination, excess_material, foreign_objects	21	26
	crack	5	crack, cracked	26	26
	faulty_imprint	5	faulty_imprint, faulty_text	19	26
	pill_type	6	different_pill, pill_type, wrong_pill	9	26
	scratch	7	cut, scratch, scratched, scratches	24	26
screw		27		119	41
	manipulated_front	5	damaged_screw, malformed_screw	24	41
	scratch_head	5	scratch, scratched_screw	24	41
	scratch_neck	5	scratch, scratched_screw	25	41
	thread_side	6	broken_thread, damaged_screw, damaged_thread	23	41
	thread_top	6	broken_thread, damaged_screw, damaged_thread	23	41
tile		31		84	33
	crack	6	crack, cracked, cracks	17	33
	glue_strip	5	glue_strip, tape	18	33
	gray_stroke	6	big_gray_stroke, gray_stroke, grey_stroke	16	33
	oil	8	liquid, oil, oil_drop, oil_drops, oil_film	18	33

Continuation of Table 1					
category	error type	nr. of prompts	specific prompts	nr. of faulty images	nr. of good images
toothbrush	rough	6	rough, roughened_surface, scratch	15	33
	defective	5	defective, defective_toothbrush	30	12
	defective	5	defective, defective_toothbrush	30	12
transistor		20		40	60
	bent_lead	5	bent_lead, damaged_lead	10	60
	cut_lead	5	cut_lead, damaged_lead	10	60
	damaged_case	5	damaged_case, damaged_transistor	10	60
	misplaced	5	misplaced_transistor, missing_transistor	10	60
	misplaced	5	misplaced_transistor, missing_transistor	10	60
wood		23		60	19
	color	6	color, color_stains, stains	8	19
	hole	6	crack, drilled_hole, hole	10	19
	liquid	6	drops, liquid, spilt_liquid	10	19
	scratch	5	scratch, scratched_surface	21	19
zipper		38		119	32
	broken_teeth	8	broken_teeth, broken_tooth, broken_zipper, faulty_zipper, split_teeth	19	32
	fabric_border	5	cut, damaged_fabric	17	32
	fabric_interior	5	damaged_fabric, damaged_surface	16	32
	rough	7	rough_edges, rough_teeth, rough_tooth, rough_zipper	17	32
	split_teeth	7	broken_teeth, broken_zipper, faulty_zipper, split_teeth	18	32
	split_teeth	7	broken_teeth, broken_zipper, faulty_zipper, split_teeth	18	32
	split_teeth	7	broken_teeth, broken_zipper, faulty_zipper, split_teeth	18	32
	split_teeth	7	broken_teeth, broken_zipper, faulty_zipper, split_teeth	18	32
	split_teeth	7	broken_teeth, broken_zipper, faulty_zipper, split_teeth	18	32

Continuation of Table 1					
category	error type	nr. of prompts	specific prompts	nr. of faulty images	nr. of good images
	squeezed_teeth	6	broken_teeth, broken_zipper, squeezed_teeth	16	32
End of Table					

3.1 Prompts

For each error type, I tried to think of some fitting prompts that could catch the given errors. An important aspect when writing prompts was to include some similar ones, changing only one word, or the form of the word such as using singular or plural. One prompt was the respective name of the error type in the MVTEC dataset, as that should be a fitting description. Additionally I wanted to see how well more general prompts can perform, so I also ran "damage", "damaged", "broken" and "error" for every type at first. Later on I dropped "error" as a prompt as it barely got any results at all, even compared to the other three.

3.2 Running the Model

As the first step I wrote all the prompts in a .json file containing an entry for each category and error type of the MVTEC dataset. The model was then run with each prompt individually, returning predicted masks, corresponding certainties and anomaly maps. Figure 5 shows the pipeline for one error type. Since SAM 3 only returns anomaly maps and masks if there was an object found with a certainty as high as the given threshold, the confidence threshold was set to 0.25, as that was a good tradeoff between filtering out noise and still including most relevant findings. In order to get a single mask and anomaly map per image, the found masks were joined together and anomaly maps were also joined, with the highest value per pixel being chosen. Since the evaluation also needs good pictures to check for false positives, for each run, the "good" folder of the corresponding category was run through the model. In the case of the good pictures, blank ground truth masks were created, to make the evaluation in the next step possible. The results of the segmentation from SAM 3 were then further used for visualization, but more importantly to calculate metrics with anomalib in the last step.

3.3 Evaluation

To compare the performance of each prompt, different metrics were calculated. While pixel-level AUROC (Area Under the Receiver Operating Characteristic Curve) and AUPRO (Area Under Per Region Overlap) scores were also noted,

primarily the image-level AUROC scores were used to create graphics. I decided to choose this metric, as it shows how well the model can differentiate the bad images from the bad ones, which is the main focus of this work. A deviation in the predicted masks is not as important in practice as finding errors in the first place. For the calculation, the integrated evaluation functions in anomalib [Akçay et al. 2022] were used.

The ROC curve plots the true positive rate against the false positive rate across all threshold values. The area under the curve is always a value from 0 to 1, indicating the probability of a model ranking a positive example higher than a negative one. In my case that would be an image with an anomaly getting a higher score than one without an anomaly. For this metric 1 is the best score possible, while 0.5 indicates the same performance as assigning random labels. Values close to 0 could stem from some misinterpretation, as the model would distinguish good and bad well, but assign inverted labels. The image-level AUROC just evaluates the classification of the whole image and the associated score, while the pixel-level AUROC evaluates the score for each pixel.

The area under the ROC curve is calculated via trapezoidal integration, shown in Equation 1, with α being the false positive rate and $1 - \beta$ being the true positive rate [Bradley 1997, p. 1146].

$$\text{AUC} = \sum_i \left\{ (1 - \beta_i \cdot \Delta\alpha) + \frac{1}{2} [\Delta(1 - \beta) \cdot \Delta\alpha] \right\} \quad (1)$$



PRO takes each region of the ground truth into consideration separately, making smaller anomalous regions as important as bigger ones. In other, pixel based metrics the score get heavily influenced by big anomalies, only getting a minimally lower score if small ones are missed.

$$\text{PRO} = \frac{1}{N} \sum_i \sum_k \frac{|P_i \cap C_{i,k}|}{|C_{i,k}|} \quad (2)$$

Equation 2 shows how the PRO for a certain threshold is calculated, with k being a connected component of the ground truth i , $C_{i,k}$ being the anomalous pixels and P_i the predicted anomalous pixels [Bergmann et al. 2021, p. 1045]

4 Results

As expected, the results varied a lot between prompts and categories, with some performing well and showing good results, and some not showing any at all. In a few cases they even performed either worse than a random classifier would or showed unexpected behavior.

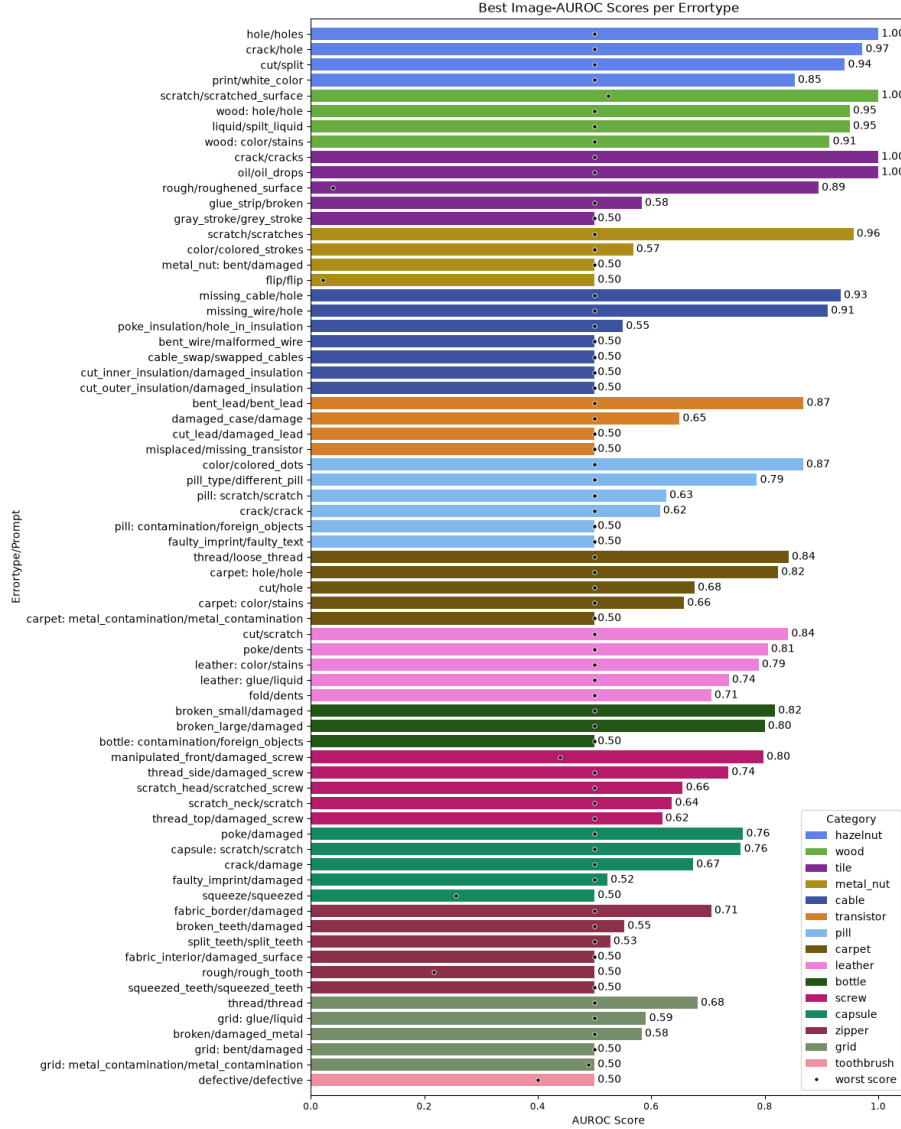


Figure 6: The best image AUROC results of each category and error type in the dataset, sorted by highest single result.

Table 2: Count of error types where the best prompt got an AUROC score within the given intervals.

	bad score (≤ 0.70)	decent score (0.70 - 0.85)	good score (> 0.85)
count of error types	38	16	15

Looking at figure 6, there are four prompts that scored the best possible value of 1.0. The categories where SAM 3 performed the best overall were "hazelnut" and "wood" in which for every error type at least one prompt could score a value of 0.8 and 0.9, respectively. In the "toothbrush" category not a single prompt performed better than 0.5, meaning a random classifier would be as effective. Also "grid" and "zipper" do not have any particularly good results to show for, with most being close to 0.5. While "leather" had consistently average results, other categories such as "metal nut" or "cable" got mostly low values, with one or two relatively high ones.

Table 2 shows that almost half of all error types got at least one score over 0.70 with 31 out of 69 total types, and just above every fifth managed to get a good score of over 0.85 with 15 out of 69.

As for the worst scores of each error type, most had a value of 0.5, meaning at least one prompt used could not correctly classify the images at all. However, that was not too surprising, as some prompts were intentionally vague so they could be used for all categories. Additionally, some errors are hard to accurately describe with a text prompt and some are simply not possible to find using a zero shot approach.

Interesting results are the ones with a significantly lower score than 0.5, essentially ranking good images higher than faulty ones in terms of error certainty. The lowest recorded image-AUROC score was 0.0217 in the category "metal nut", error type "flip", using the prompt "flipped nut".

4.1 Generic Prompts

After general performance, I also wanted to find out what results the generic prompts used across categories and error types yielded.

The highest result managed to achieve a respectable AUROC score of 0.94, as seen in Table 3. This makes it part of the best results overall. Five more scored between 0.80 and 0.90 and across the three different prompts, 41 scored higher than 0.50. It is worth to note, that only five of those were from the prompt "broken", with the best result being 0.58, making it clearly the worst of the three.

With "damage" and "damaged" being very similar prompts I expected similar results and wanted to compare them. Looking at Table 4, "damaged" yielded slightly better results. To accurately compare them, both AUROC scores for

Table 3: The 25 error types with the best performing generic prompts.

Category	Error type	Prompt	AUROC Score
hazelnut	crack	damaged	0.94
hazelnut	crack	damage	0.89
bottle	broken_small	damage	0.82
bottle	broken_small	damaged	0.82
hazelnut	hole	damaged	0.81
bottle	broken_large	damaged	0.80
capsule	poke	damaged	0.76
bottle	broken_large	damage	0.75
capsule	poke	damage	0.71
zipper	fabric_border	damaged	0.71
wood	hole	damaged	0.70
hazelnut	hole	damage	0.69
leather	cut	damaged	0.68
capsule	crack	damage	0.67
capsule	crack	damaged	0.65
transistor	damaged_case	damage	0.65
carpet	cut	damaged	0.65
leather	cut	damage	0.63
transistor	damaged_case	damaged	0.60
grid	broken	damaged	0.58
tile	glue_strip	broken	0.58
wood	scratch	damaged	0.57
carpet	hole	damaged	0.56
zipper	fabric_border	damage	0.56
hazelnut	cut	damaged	0.56

each error type were recorded in Table 5. With a few exceptions, "damaged" seems to have a slightly higher or at least even score. The biggest difference was 0.20 for the error type "wood/hole", while the biggest difference in favor of "damage" was only 0.05 for "transistor/damaged_case".

Table 4: Count of how often each prompt got an AUROC score within the given intervals.

Prompt	0.5	(0.5, 0.6]	(0.6, 0.7]	(0.7, 0.8]	(0.8, 0.9]	(0.9, 1]
broken	64	5	0	0	0	0
damage	52	9	4	2	2	0
damaged	50	8	5	2	3	1

Table 5: Comparison between results of prompts "damage" and "damaged", best 20, sorted by results of "damaged".

Category/Error type	Prompt	
	damage	damaged
hazelnut/crack	0.89	0.94
bottle/broken_small	0.82	0.82
hazelnut/hole	0.69	0.81
bottle/broken_large	0.75	0.80
capsule/poke	0.71	0.76
zipper/fabric_border	0.56	0.71
wood/hole	0.50	0.70
leather/cut	0.63	0.68
capsule/crack	0.67	0.65
carpet/cut	0.53	0.65
transistor/damaged_case	0.65	0.60
grid/broken	0.50	0.58
wood/scratch	0.52	0.57
hazelnut/cut	0.53	0.56
carpet/hole	0.50	0.56
zipper/broken_teeth	0.53	0.55
metal_nut/color	0.50	0.52
capsule/faulty_imprint	0.52	0.52
screw/scratch_neck	0.52	0.52
bottle/contamination	0.50	0.50

4.2 Visualizations



Figure 7: Image 000 from category "hazelnut", error type "print" together with the anomaly map, predicted masks and ground truth masks, respectively. The prompt used was "white color".

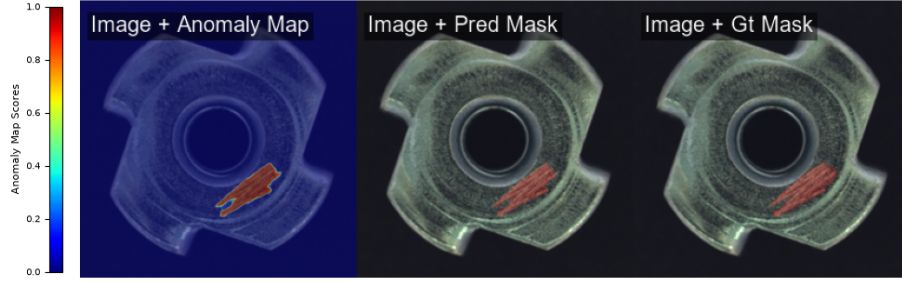


Figure 8: Image 000 from category "metal_nut", error type "scratch" together with the anomaly map, predicted masks and ground truth masks, respectively. The prompt used was "scratches".

Figure 7 and Figure 8 show visualizations of some of the better results, where the predicted mask comes very close to the ground truth mask. Across the error type "print", the pixel level AUROC score was 0.8317, for type "scratch" it was 0.8854, using the prompts "white color" and "scratches", respectively. These values are a bit lower than the corresponding image level AUROC, which is to be expected. However, as can be seen in the figures, the errors can be clearly identified from the masks.

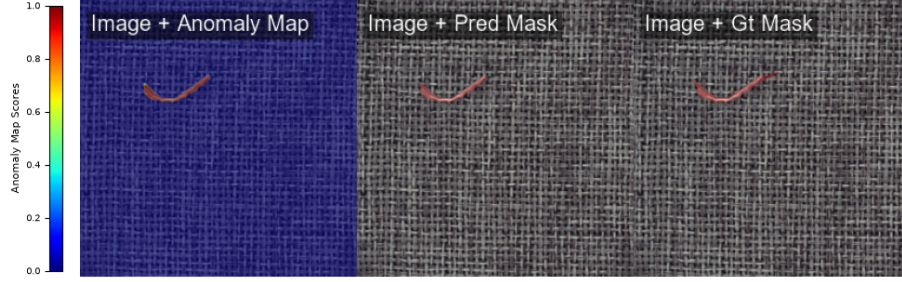


Figure 9: Image 000 from category "carpet", error type "thread" together with the anomaly map, predicted masks and ground truth masks, respectively. The prompt used was "loose thread".

While Figure 9 also shows a case of accurate predicted masks, the interesting part here is the comparison of the AUROC scores. With an image level score of 0.8421, the pixel level score is actually higher with 0.8806 for the error type "thread" using the prompt "loose thread". This means for this specific case, the model sometimes struggles to detect the error, but when it does, the masks are closer to the ground truth masks than for most other error types.

A high image AUROC score does not always mean that the model is actually doing what we want it to. One example for this in my experiment is the error type "scratch" in the category "wood" with the prompt "scratched surface". While the calculated image AUROC score is 1.0, the highest possible value, after looking at the predicted masks, it becomes clear that the prompt does not actually find the defects I was looking for.



Figure 10: Image 000 from category "wood", error type "scratch" together with the anomaly map, predicted masks and ground truth masks, respectively. The prompt used was "scratched surface".

Here it can be clearly seen that this is not the wanted result where the model only marks the parts with scratches, but rather the whole image. In the anomaly map, the scratched parts are even given a lower confidence. This gives

the impression that the prompt was not interpreted as intended and focused more on the "surface" part, ignoring the "scratched" before that.

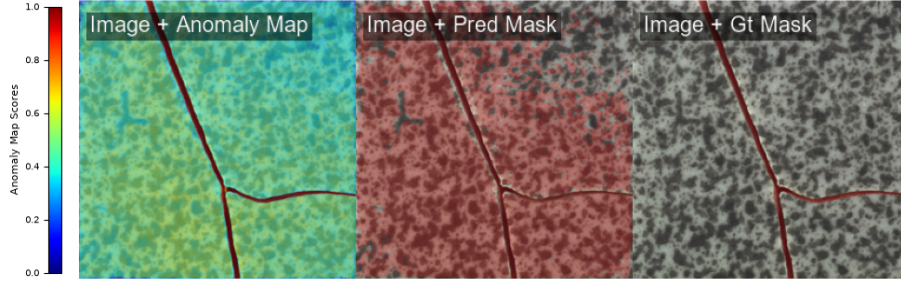


Figure 11: Image 000 from category "tile", error type "crack" together with the anomaly map, predicted masks and ground truth masks, respectively. The prompt used was "cracks".

In some cases the model correctly classified images, but also marked parts that are not faulty and do not fit the prompt. In figure 11 part of the texture is ascribed a non-zero certainty, in this case high enough to be part of the predicted mask. However, the big crack in the middle scored a much higher certainty, indicating a good performance with some noise.

4.3 Notable Results

With the text on all the hazelnuts in the error type "print" being white, I tried out both the prompts "letters" and "white letters", assuming the latter one would perform better, or at least the same. However, it turned out to yield slightly worse results, with an image AUROC of 0.5882 for "letters" and 0.5588 for "white letters".

Some errors can be hard or near impossible to find with zero shot anomaly detection, including error types like "misplaced_transistor", "cable_swap", "missing_cable" or "missing_wire". Without prior training on good images, the model can not know the specifics of what the product is supposed to look like and where a transistor or cable should be. There are different approaches that can work though. By looking at the consequences of the specific errors, for example a missing cable or wire leaving a hole in the space where it should be. Using this approach and prompting error types "missing_cable" and "missing_wire" with "hole" resulted in an image AUROC score of 0.9339 and 0.9112, respectively. While those are high values, this way of prompting might not be suitable for all use cases, due to making it hard to correctly classify the type of error, or some products having holes by design.

Errors like "cable_swap" could potentially also be found, by prompting for each color of cables that are needed, classifying the image as bad if one of those was not found. This is dependent on how well the model performs at finding

the differently colored cables, of course. It would also need a whole different framework which would need to be tested and evaluated.

5 Discussion

Overall it can be said, that using SAM 3 with text prompts for anomaly detection can work, however it still has some clear weaknesses and shortcomings that would need to be further worked on. Judging by the results of my experiments, this can be a helpful tool in addition to other methods or an alternative if a zero shot model is needed. But also the goal of this work was not optimizing prompts for each error class, meaning there a lot of room for improvement in those with bad scores. With the top results already competing with other state-of-the-art models like PatchCore, which, using the anomalib implementation, has a mean image level AUROC score of 0.981 across the MVTec dataset, ranging from 0.934 to 1.000 per category [Akçay et al. 2022], there is a lot of potential. A possible approach to improve results could be to run multiple prompts on each image, combining the masks. This could solve the issue of hard to find catch-all prompts, making it possible to detect different types of defects. This is also how this method would be used in practice, as in the real world all different kinds of errors can occur simultaneously. A set of prompts would need to be defined, covering most or all error types that most commonly occur for a certain product. To save some resources, if one error is found for an object, the other prompts would not need to be run, as the product can be sorted out with the first error. In some cases the frequency of errors might be known, making it possible to start with prompts for the most common ones for more efficiency. In the long run, this is also data that can be acquired from this method, by counting the number of errors found per prompt.

References

- Akçay, Samet, Dick Ameln, Ashwin Vaidya, Barath Lakshmanan, Nilesh Ahuja, et al. (2022). “Anomalib: A deep learning library for anomaly detection”. In: *2022 IEEE International Conference on Image Processing (ICIP)*. IEEE, pp. 1706–1710.
- Aqeel, Muhammad, Shakiba Sharifi, Marco Cristani, and Francesco Setti (2024). “Self-supervised iterative refinement for anomaly detection in industrial quality control”. In: *arXiv preprint arXiv:2408.11561*.
- Bergmann, Paul, Kilian Batzner, Michael Fauser, David Sattlegger, and Carsten Steger (Apr. 2021). “The MVTec Anomaly Detection Dataset: A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection”. In: *International Journal of Computer Vision* 129. DOI: 10.1007/s11263-020-01400-4.

- Bolya, Daniel, Po-Yao Huang, Peize Sun, Jang Hyun Cho, Andrea Madotto, et al. (2025). *Perception Encoder: The best visual embeddings are not at the output of the network*. arXiv: 2504.13181 [cs.CV]. URL: <https://arxiv.org/abs/2504.13181>.
- Bradley, Andrew P. (1997). “The use of the area under the ROC curve in the evaluation of machine learning algorithms”. In: *Pattern Recognition* 30.7, pp. 1145–1159. ISSN: 0031-3203. DOI: [https://doi.org/10.1016/S0031-3203\(96\)00142-2](https://doi.org/10.1016/S0031-3203(96)00142-2). URL: <https://www.sciencedirect.com/science/article/pii/S0031320396001422>.
- Carion, Nicolas, Laura Gustafson, Yuan-Ting Hu, Shoubhik Debnath, Ronghang Hu, et al. (2025). *SAM 3: Segment Anything with Concepts*. arXiv: 2511.16719 [cs.CV]. URL: <https://arxiv.org/abs/2511.16719>.
- Li, Shengze, Jianjian Cao, Peng Ye, Yuhan Ding, Chongjun Tu, et al. (2024). *ClipSAM: CLIP and SAM Collaboration for Zero-Shot Anomaly Segmentation*. arXiv: 2401.12665 [cs.CV]. URL: <https://arxiv.org/abs/2401.12665>.
- Peng, Yun, Xiao Lin, Nachuan Ma, Jiayuan Du, Chuangwei Liu, et al. (2025). “SAM-LAD: Segment Anything Model meets zero-shot logic anomaly detection”. In: *Knowledge-Based Systems* 314, p. 113176. ISSN: 0950-7051. DOI: <https://doi.org/10.1016/j.knosys.2025.113176>. URL: <https://www.sciencedirect.com/science/article/pii/S0950705125002230>.
- Rajendran, T., N Mohamed Imtiaz, K Jagadeesh, and Balaji Sampathkumar (2024). “Cybersecurity Threat Detection Using Deep Learning and Anomaly Detection Techniques”. In: *2024 International Conference on Knowledge Engineering and Communication Systems (ICKECS)*. Vol. 1, pp. 1–7. DOI: 10.1109/ICKECS61492.2024.10617347.
- Tschuchnig, Maximilian E and Michael Gadermayr (2021). “Anomaly detection in medical imaging-a mini review”. In: *International Data Science Conference*. Springer, pp. 33–38.
- Yang, Hui-Yue, Hui Chen, Ao Wang, Kai Chen, Zijia Lin, et al. (2024). *Promptable Anomaly Segmentation with SAM Through Self-Perception Tuning*. arXiv: 2411.17217 [cs.CV]. URL: <https://arxiv.org/abs/2411.17217>.
- Zakaria, Rafi Muhammad, Mohammad Mahmudur Rahman, Hasibur Rahman, Mainuddin Adel Rafi, Anisuzzaman Minto, et al. (2025). “Detecting financial fraud in real-time transactions using graph neural networks and anomaly detection techniques”. In: *Journal of Economics, Finance and Accounting Studies* 7.6, pp. 01–13.